

UNIDAD DE TRABAJO 01 · PERSISTENCIA BÁSICA

# Manejo de Ficheros en Java

Fundamentos teóricos y técnicos de Entrada/Salida: Ficheros de Texto Plano, API Java NIO.2, Ficheros Binarios Tipados, RandomAccessFile, Serialización de Objetos y Procesamiento XML

MÓDULO CURRICULAR

**Acceso a Datos (0486)**

DOCENTE

**Sergio Capdevila Díez**

CICLO FORMATIVO

**2º DAM (Desarrollo Multiplataforma)**

DURACIÓN ESTIMADA

**30 Horas Lectivas (Trimestre 1)**

RESULTADOS DE APRENDIZAJE

**RA1 (CE1.a, CE1.b, CE1.c, CE1.d, CE1.e)**

ENTORNO TECNOLÓGICO

**Java SE 21 / Eclipse IDE / Maven**

# Los 3 Grandes Bloques de Persistencia

De los formatos legibles por humanos a las estructuras binarias y documentos estructurados

## 1. Ficheros de Texto

BLOQUE  
1

- Codificación de caracteres (UTF-8 vs ISO-8859-1).
- Jerarquía Reader / Writer con Buffers.
- La moderna API `java.nio.file` (Java 11+).
- Streams funcionales para logs masivos (`Files.lines`).

## 2. Ficheros Binarios

BLOQUE  
2

- Flujos de datos primitivos (`DataStream`).
- Acceso aleatorio con `RandomAccessFile` (RAF).
- Serialización nativa de objetos (`Serializable`).
- **Solución al append:** `MultiObjectOutputStream`.

## 3. Ficheros XML

BLOQUE 3

- Árbol en memoria RAM: **DOM** + Transformer.
- Procesamiento por eventos reactivos: **SAX**.
- Cursor iterativo bajo demanda: **StAX**.
- Mapeo objeto-documento declarativo: **JAXB**.

# ¿Por qué Texto Plano y qué es un Charset?

La traducción crítica entre los caracteres de la RAM y los bytes del disco

## El Ecosistema del Texto

- **Interoperabilidad total:** Legible por personas, scripts bash, herramientas de monitorización y cualquier lenguaje.
- **Estructura secuencial:** Cadenas separadas por saltos de línea (`\n` en Linux/macOS, `\r\n` en Windows).
- **RAM vs Disco:** Java gestiona cadenas en memoria en **UTF-16** (2 bytes/carácter). Al escribir en disco, debe serializarlas a bytes mediante un **Charset**.

## ⚠ Trampa Clásica de Aula: El Charset por Defecto

Instanciar `new FileReader("datos.txt")` sin especificar charset usa la codificación de la máquina anfitriona (en Windows puede ser `windows-1252`).

Si ese archivo se sube a un contenedor Docker o servidor Linux con UTF-8, las tildes y eñes se corromperán inmediatamente (**Mojibake**).

**Regla de Oro:** Especificar `StandardCharsets.UTF_8` siempre.

## 💻 Práctica en Vivo en Aula: Actividad 1.1

Abre tu IDE y completa la **Actividad 1.1** del Cuaderno de Aula (conversión de fichero ISO-8859-1 a UTF-8).

# El Patrón Decorador y los Buffers

Minimizando llamadas de E/S al sistema operativo para multiplicar el rendimiento

## La Cadena de Flujos (Streams)

- **Flujo base:** `FileReader` lee caracteres físicos del disco.
- **Flujo decorador:** `BufferedReader` añade una memoria intermedia (buffer de 8 KB).
- **Método clave:** `readLine()` devuelve la línea completa como `String` y retorna `null` al llegar al final (EOF).

```
try (BufferedReader br = new BufferedReader( new
FileReader("app.log", StandardCharsets.UTF_8))) {
String linea; while ((linea = br.readLine()) != null) {
System.out.println(linea); } }
```

## Analogía del Mundo Real: La Carretilla vs El Camión

Pedir a un operario que vaya al almacén central a buscar **un único ladrillo** cada vez (un carácter con `read()`) es ineficiente y satura el tráfico.

El `BufferedReader` es un **camión de 8.000 ladrillos**: hace un solo viaje al almacén (disco) para cargar miles de caracteres en RAM. Tu código va tomando ladrillos de esa memoria instantáneamente sin pausar la CPU.

## Práctica en Vivo en Aula: Actividad 1.2

Abre tu IDE y completa la **Actividad 1.2** del Cuaderno de Aula (lectura y filtrado con `BufferedReader` y `BufferedWriter`).

# Escritura: **BufferedWriter** y **PrintWriter**

Rendimiento puro frente a potencia de formateo para informes y tablas

## BufferedWriter

ALTO RENDIMIENTO

- Buffer de memoria intermedio para escrituras en bloque.
- Método `write(String)` para emitir cadenas.
- Método `newLine()`: inserta el salto de línea multiplataforma adecuado del sistema operativo.
- Vacía el buffer con `flush()` antes de cerrar.

## PrintWriter

FORMATEO AVANZADO

- Permite imprimir con `print()`, `println()` y `printf()`.
- Acepta argumentos numéricos sin conversiones manuales a texto.
- No lanza excepciones de I/O en cada llamada (usa `checkError()`).
- Ideal para generar reportes tabulares, facturas y ficheros CSV.

### Práctica en Vivo en Aula: Actividad 1.3

Turno en el IDE: desarrolla la **Actividad 1.3** del Cuaderno de Aula (generador de factura con `PrintWriter.printf()`).

# La API Moderna Java NIO.2: Paths y Files

Operaciones atómicas, mayor seguridad y código conciso en Java 11+

## Operaciones en Memoria Instantáneas

- `Path.of("dir", "config.cfg")`: Representación moderna e inmutable de rutas.
- `Files.writeString(path, "texto", charset, opciones)`: Escribe una cadena completa en un solo paso.
- `Files.readString(path, charset)`: Lee el fichero completo como un único `String`.
- `Files.readAllLines(path, charset)`: Carga todas las líneas en un `List<String>`.

## Ejemplo de Uso Práctico

```
Path ruta = Path.of("datos", "servidor.properties"); //  
Escritura atómica directa Files.writeString(ruta,  
"puerto=8080\nmodo=prod", StandardCharsets.UTF_8,  
StandardOpenOption.CREATE,  
StandardOpenOption.TRUNCATE_EXISTING); // Lectura  
directa a String String config = Files.readString(ruta,  
StandardCharsets.UTF_8); System.out.println(config);
```

### Práctica en Vivo en Aula: Actividad 1.4

Turno en el IDE: desarrolla la **Actividad 1.4** del Cuaderno de Aula (persistencia atómica con `Files.writeString()` y `Files.readString()`).

# Procesamiento Masivo: Files.lines()

Java Streams con evaluación perezosa para ficheros gigantescos

## ★ El Peligro de readAllLines() en Producción

Si abres un log de servidor de **4 Gigabytes** con `Files.readAllLines()`, la JVM intentará instanciar millones de objetos `String` a la vez en el Heap de memoria RAM.

**Resultado:** `java.lang.OutOfMemoryError: Java heap space` y caída inmediata del servicio.

## La Solución: Files.lines() (Lazy Stream)

Lee las líneas del disco bajo demanda, una por una, a medida que el pipeline del Stream las va consumiendo:

```
// Requiere try-with-resources para cerrar el flujo de disco
try (Stream<String> lineas = Files.lines(Path.of("server.log"))) {
    long numCriticos = lineas.filter(l -> l.contains("[FATAL]"))
        .peek(System.out::println).count();
    System.out.println("Total incidencias: " + numCriticos);
}
```

## 🖥️ Práctica en Vivo en Aula: Actividad 1.5

Turno en el IDE: desarrolla la **Actividad 1.5** del Cuaderno de Aula (filtrado perezoso de logs con `Files.lines()`).

# ¿Por qué Binario? Eficiencia y Estructura

Almacenamiento en representación nativa de máquina sin sobrecoste de parseo

## Texto vs Binario: Caso Práctico

Supongamos que guardamos el número entero 2147483647:

- **En texto plano:** Ocupa 10 caracteres (10 bytes en UTF-8 o 20 en UTF-16). Para leerlo, la CPU debe gastar ciclos ejecutando `Integer.parseInt()`.
- **En binario puro:** Ocupa exactamente **4 bytes** (32 bits). La lectura es una copia directa de bytes a memoria.

## 💡 Ventajas Clave del Formato Binario

- **Ahorro de espacio:** Compresión natural de datos numéricos y booleanos.
- **Velocidad extrema:** Transferencia de datos bus-a-bus sin conversiones de caracteres.
- **Tipado estricto:** Se conservan los tipos de datos primitivos de Java sin ambigüedades.



## Práctica en Vivo en Aula: Actividad 2.1

Abre tu IDE y completa la **Actividad 2.1** del Cuaderno de Aula (escritura binaria con `DataOutputStream`).

# Datos Primitivos: DataStreams

DataOutputStream y DataInputStream para persistencia tipada multiplataforma

## Métodos Específicos por Tipo

- `writeInt(int)`: Escribe exactamente 4 bytes binarios.
- `writeDouble(double)`: Escribe exactamente 8 bytes binarios.
- `writeBoolean(boolean)`: Escribe 1 byte (0x01 / 0x00).
- `writeUTF(String)`: Escribe la longitud del texto (2 bytes) seguida de los caracteres UTF-8.

## ⚠ El Contrato Estricto de los Flujos de Datos

Los ficheros binarios no tienen etiquetas ni comas separadoras.

**Regla Fundamental:** Debes leer los datos **exactamente en el mismo orden y con el mismo método** con el que fueron escritos. Si escribiste `writeInt` y luego `writeDouble`, pero intentas leer primero `readDouble`, los bytes se desfazarán y obtendrás datos totalmente corruptos.



## Práctica en Vivo en Aula: Actividad 2.1

Turno en el IDE: desarrolla la **Actividad 2.1** del Cuaderno de Aula (persistencia de telemetría de sensores en binario).

# Trampa de Examen: Control de Fin con EOFException

Por qué `readInt()` no puede devolver `-1` ni `null` al terminar

## El Problema de los Métodos Tipados

En flujos de texto, `readLine()` devuelve `null`. Pero en `dis.readInt()`, el valor `-1` es un entero válido (`0xFFFFFFFF`) que podría ser un dato real de la aplicación.

Por tanto, al alcanzar el fin del fichero, Java lanza **`EOFException`**.

## Estructura Correcta de Bucle

```
try (DataInputStream dis = new DataInputStream( new
FileInputStream("sensores.dat"))) { while (true) { int
id = dis.readInt(); double temp = dis.readDouble();
System.out.printf("[%d] %.2f °C\n", id, temp); } }
catch (EOFException e) { // Fin natural y previsto del
archivo System.out.println("Fin de lectura
alcanzado."); } catch (IOException e) {
System.err.println("Error de lectura: " +
e.getMessage()); }
```



### Práctica en Vivo en Aula: Actividad 2.2

Turno en el IDE: desarrolla la **Actividad 2.2** del Cuaderno de Aula (bucle infinito controlado con captura de `EOFException`).

# Acceso Aleatorio: RandomAccessFile (RAF)

Tratando el disco como un inmenso array de bytes indexable

## El Puntero de Archivo (File Pointer)

- Los flujos convencionales son **secuenciales** (para leer el dato 1000 hay que leer los 999 anteriores).
- RandomAccessFile permite saltar a cualquier byte arbitrario mediante el método seek(long posicionBytes).
- getFilePointer(): Retorna la posición actual del cursor.
- length(): Retorna el tamaño total en bytes del fichero (con setLength(n) podemos truncarlo o extenderlo).
- skipBytes(int n): Salta n bytes hacia adelante sin necesidad de leerlos.
- Modos de apertura: "r" (solo lectura) y "rw" (lectura y escritura).



## Analogía: Cinta de Casete vs Tocabiscos

Un flujo secuencial (InputStream) es una **cinta de casete**: si quieres escuchar la canción 5, debes rebobinar o avanzar pasando obligatoriamente por las canciones 1, 2, 3 y 4.

RandomAccessFile es el **brazo de un tocadiscos o un CD**: levantas la aguja y la colocas con precisión milimétrica (seek) directamente en la pista que quieres reproducir o sobrescribir.



## Práctica en Vivo en Aula: Actividad 2.3 (Parte 1)

Abre tu IDE y comienza la **Actividad 2.3** del Cuaderno de Aula (creación de fichero de registros con RandomAccessFile).

# Registros de Longitud Fija en RAF

La base matemática para modificar registros directamente en disco

## Diseño de Registro de Empleado (36 Bytes Fijos)

id (int: 4B) + apellido (10 chars con writeChars: 20B) + departamento (int: 4B) + salario (double: 8B) = **36 Bytes**.

```
public static void actualizarSalarioDirecto(String ruta, int indiceRegistro, double nuevoSalario) { try (RandomAccessFile raf = new RandomAccessFile(ruta, "rw")) { // Cálculo matemático del byte exacto donde empieza el salario: // offset = (registro * 36) + (4 de id + 20 de apellido + 4 de dep) = base + 28 long offset = ((long) indiceRegistro * 36) + 28; raf.seek(offset); raf.writeDouble(nuevoSalario); // Sobrescribe SOLO los 8 bytes del salario System.out.println("Salario actualizado en disco en posición: " + offset); } catch (IOException e) { ... } }
```

### Práctica en Vivo en Aula: Actividad 2.3 (Parte 2)

Turno en el IDE: completa la **Actividad 2.3** del Cuaderno de Aula (modificación in-situ de salarios calculando offsets exactos).

# Serialización de Objetos en Java

Persistiendo el estado de instancias completas de la memoria RAM al disco

## 1. Serializable

Interfaz marcadora (sin métodos).  
Notifica a la JVM que autorizamos la persistencia. Si no está implementada, lanza `NotSerializableException`.

## 2. serialVersionUID

Identificador de versión. Definir siempre `private static final long serialVersionUID = 1L;` para evitar que la JVM lance `InvalidClassException` al cambiar la clase.

## 3. transient

Modificador para campos que **no deben guardarse** (contraseñas, tokens de sesión o sockets de red). Al deserializar toman su valor por defecto (`null` o `0`).

```
// Guardar: try (ObjectOutputStream oos = new ObjectOutputStream(fos)) { oos.writeObject(alumno); oos.flush(); } // Cargar: try (ObjectInputStream ois = new ObjectInputStream(fis)) { Alumno a = (Alumno) ois.readObject(); } // Tip Pro: oos.reset() limpia la caché interna de referencias si modificas un objeto antes de volver a escribirlo
```



### Práctica en Vivo en Aula: Actividad 2.4

Turno en el IDE: desarrolla la **Actividad 2.4** del Cuaderno de Aula (serialización de usuarios con campos `transient` y control de `serialVersionUID`).

# El Problema del Append: StreamCorruptedException

La trampa de añadir objetos a un archivo existente con `FileOutputStream(ruta, true)`

## ✦ ¿Qué sucede bajo el capó?

Al instanciar un `ObjectOutputStream`, su constructor escribe automáticamente una **cabecera de 4 bytes** (`STREAM_MAGIC = 0xACED` y `STREAM_VERSION = 0x0005`).

Si abres en modo `append = true`, se escribe una **segunda cabecera intermedia** en mitad del archivo.

Al leer con `ObjectInputStream`, este procesa el primer objeto, y al llegar al segundo lote se encuentra la cabecera `0xACED` donde esperaba un código de objeto (`0x73`), lanzando:

`StreamCorruptedException: invalid type code: AC`

## Esquema Binario de Corrupción

```
Archivo en disco tras dos escrituras independientes con
append: [ AC ED 00 05 ] → Cabecera inicial (correcta)
[ OBJETO 1 ] → Objeto leído correctamente [ AC ED 00
05 ] → ¡CABECERA DUPLICADA EN MITAD! ▲ └─ ¡CRASH!
ObjectInputStream revienta aquí [ OBJETO 2 ]
```

## 🖥️ Práctica en Vivo en Aula: Actividad 2.5 (Parte 1)

Reproduce en tu IDE el fallo de `append` con `FileOutputStream(ruta, true)` y comprueba el lanzamiento de `StreamCorruptedException`.

# Solución Docente: MiObjectOutputStream

Sobrescribir writeStreamHeader() para anular cabeceras secundarias

## 1. Crear la Subclase Sin Cabecera

```
class MiObjectOutputStream extends ObjectOutputStream {  
    public MiObjectOutputStream(OutputStream out) throws  
        IOException { super(out); } @Override protected void  
    writeStreamHeader() throws IOException { // Al dejar  
        este método vacío, NO se escribe // la cabecera  
        0xACED0005 intermedia } }
```

## 2. Lógica de Persistencia Append

```
File f = new File("datos.dat"); boolean  
yaExisteConDatos = f.exists() && f.length() > 0; try  
(FileOutputStream fos = new FileOutputStream(f, true);  
ObjectOutputStream oos = yaExisteConDatos ? new  
MiObjectOutputStream(fos) : new  
ObjectOutputStream(fos)) {  
    oos.writeObject(nuevoAlumno); }
```

### Práctica en Vivo en Aula: Actividad 2.5 (Parte 2 - El Gran Reto)

Turno en el IDE: resuelve la **Actividad 2.5** del Cuaderno de Aula utilizando tu `MiObjectOutputStream` para persistir incrementalmente sin corrupción.

# Ficheros XML: El Ecosistema de Parsers

El estándar de facto para documentos estructurados y las 4 formas de procesarlo

## Características de XML

- **Jerarquía estricta:** Un único elemento raíz que contiene ramas y hojas.
- **Auto-descriptivo:** Etiquetas semánticas comprensibles por personas y sistemas.
- **Independencia:** Estándar W3C independiente del sistema operativo y lenguaje.

## Los 4 Métodos en Java

- **DOM (W3C):** Carga el árbol completo en memoria RAM.
- **SAX:** Procesador secuencial reactivo por eventos (Push).
- **StAX:** Procesador por flujo con control del programador (Pull).
- **JAXB:** Mapeo directo y declarativo entre clases Java y XML.

### Cuaderno de Aula: Bloque 3 (Ficheros XML)

En este bloque abordaremos las **Actividades 3.1 a 3.4** para dominar DOM, Transformer, SAX y JAXB paso a paso en el IDE.

# Manera 1: DOM (Document Object Model)

El árbol completo en memoria y la trampa de los nodos de texto whitespace

## Parseo con DocumentBuilder

```
DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance(); DocumentBuilder
builder = factory.newDocumentBuilder(); Document doc =
builder.parse(new File("catalogo.xml")); //
;OBLIGATORIO! Normalizar para limpiar nodos de texto
partidos doc.getDocumentElement().normalize(); NodeList
lista = doc.getElementsByTagName("producto"); for (int
i = 0; i < lista.getLength(); i++) { Node n =
lista.item(i); if (n.getNodeType() ==
Node.ELEMENT_NODE) { Element e = (Element) n;
System.out.println(e.getAttribute("id")); } }
```

### ⚠ Trampa: Nodos de Texto Vacíos (#text)

En XML, los saltos de línea y tabuladores entre etiquetas son interpretados por DOM como nodos hijos de tipo **Text**.

Si recorres `nodo.getChildNodes()` a ciegas, fallará esperando elementos.

**Solución:** Verificar siempre `if (nodo.getNodeType() == Node.ELEMENT_NODE)`.

### Práctica en Vivo en Aula: Actividad 3.1

Turno en el IDE: realiza la **Actividad 3.1** del Cuaderno de Aula parseando `catalogo.xml`, esquivando nodos `#text` y actualizando precios con DOM.

# Creación y Volcado DOM con Transformer

Construyendo el árbol desde cero y serializándolo a disco con pretty-printing

## Flujo de Construcción y Volcado

```
Document doc = builder.newDocument(); Element raiz = doc.createElement("concesionario"); doc.appendChild(raiz); Element
coche = doc.createElement("coche"); coche.setAttribute("matricula", "1234-ABC"); Element marca = doc.createElement("marca");
marca.setTextContent("Toyota"); coche.appendChild(marca); raiz.appendChild(coche); // Volcado a disco con sangría automática
Transformer tf = TransformerFactory.newInstance().newTransformer(); tf.setOutputProperty(OutputKeys.INDENT, "yes");
tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4"); tf.transform(new DOMSource(doc), new
StreamResult(new File("concesionario.xml")));
```



### Práctica en Vivo en Aula: Actividad 3.2

Turno en el IDE: aborda la **Actividad 3.2** para construir un árbol DOM de alumnos desde cero y serializarlo con Transformer aplicando *pretty-print*.

## Manera 2: SAX (Simple API for XML)

El procesador reactivo basado en eventos (Push) para archivos masivos

### Callbacks de DefaultHandler

- **startElement():** Se dispara al encontrar una etiqueta de apertura (ej. <coche>).
- **characters():** Se dispara cuando lee texto. *¡Ojo! El parser puede llamar a este método varias veces seguidas; se debe acumular con un StringBuilder.*
- **endElement():** Se dispara al cerrar la etiqueta (ej. </coche>).

### ⚡ Consumo de Memoria Casi Nulo

SAX no construye ningún árbol en memoria RAM. Lee byte a byte hacia adelante.

**Caso de uso:** Si debes procesar un fichero de facturas de **8 GB** enviado por Hacienda, DOM colapsará con `OutOfMemoryError`. SAX lo procesará con tan solo **20 MB de RAM**.

### 💻 Práctica en Vivo en Aula: Actividad 3.3

Turno en el IDE: desarrolla la **Actividad 3.3** implementando tu propio `DefaultHandler` para computar estadísticas con SAX con 0 MB de consumo de árbol.

# Maneras 3 y 4: StAX (Pull) y JAXB (Mapeo)

Control del bucle iterativo y mapeo declarativo con anotaciones POJO

## StAX (Pull Parser)

XMLSTREAMREADER

A diferencia de SAX (Push), en StAX el programador gobierna el cursor y avanza a demanda:

```
XMLStreamReader r = XMLInputFactory.newInstance()
    .createXMLStreamReader(new FileReader("data.xml"));
while (r.hasNext()) { if (r.next() ==
XMLStreamConstants.START_ELEMENT &&
"precio".equals(r.getLocalName())) {
System.out.println("Precio: " + r.getElementText()); }
}
```

## JAXB (Jakarta XML Binding)

MAPEO ANOTADO

Mapeo declarativo POJO con `@XmlRootElement`,  
`@XmlElement` y `@XmlAttribute`:

```
JAXBContext ctx =
JAXBContext.newInstance(Producto.class); // Marshaller:
Java → XML con sangría Marshaller m =
ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(producto, new File("producto.xml")); //
Unmarshaller: XML → Java Producto p = (Producto)
ctx.createUnmarshaller().unmarshal(new
File("producto.xml"));
```

### Práctica en Vivo en Aula: Actividad 3.4

Turno en el IDE: completa la **Actividad 3.4** mapeando una entidad con anotaciones Jakarta/JAXB y probando Marshaller y Unmarshaller.

# Matriz Comparativa de Persistencia

Criterios de selección técnica para el entorno laboral y pruebas prácticas

| TECNOLOGÍA       | CONSUMO RAM      | VELOCIDAD I/O           | EDICIÓN IN-SITU        | CASO DE USO RECOMENDADO                          |
|------------------|------------------|-------------------------|------------------------|--------------------------------------------------|
| Texto / NIO.2    | Mínima (Streams) | Alta                    | No (Append/Reescribir) | Configuración, variables de entorno y logs.      |
| DataStream       | Mínima           | Ultra Rápida            | No (Secuencial)        | Telemetría, sensores IoT y tramas binarias.      |
| RandomAccessFile | Mínima           | Máxima                  | Sí (Directa con seek)  | Registros fijos, índices y motores de BD.        |
| Serialización    | Moderada         | Media-Alta              | Con Mi00S en append    | Cachés locales y sesiones de escritorio.         |
| XML DOM          | Muy Alta (~4x)   | Baja en ficheros > 20MB | Sí (en árbol RAM)      | Ficheros < 10 MB con consultas XPath.            |
| XML SAX / StAX   | Nula (~0 KB)     | Máxima                  | No (Solo lectura)      | Volcados masivos de gigabytes (Hacienda/Bancos). |
| XML JAXB         | Moderada         | Media                   | Vía objetos Java       | Intercambio empresarial B2B y servicios SOAP.    |

# Retos y Prácticas Guiadas de Aula

Aplica todo lo aprendido en los ejercicios del repositorio oficial

## Bloque 1: Texto y NIO.2

**Hoja B1:** Validación y saneamiento de CSV con charsets, auditoría forense de logs con Streams reactivos e informes tabulados con PrintWriter.

## Bloque 2: Binarios y RAF

**Hoja B2:** Telemetría con DataStreams y control EOFException, edición in-situ con RandomAccessFile y append con MiObjectOutputStream.

## Bloque 3: Ecosistema XML

**Hoja B3:** Construcción DOM con Transformer sangrado, procesamiento masivo con SAX/StAX y mapeo declarativo POJO-XML con JAXB.

### ¡Manos al Código en Eclipse IDE!

Trabaja en directo en el aula con el **Cuaderno de Actividades (14 retos guiados)** y afianza tu dominio con las **Hojas de Ejercicios por Bloques (B1, B2, B3)** y el proyecto capstone **PRAC-AD-UT1: Gestor Multi-Formato**.